20 de octubre de 2026

FastParking

Documento técnico de diseño e implementación

Requerimientos · Stack tecnológico · Arquitectura · MER · BPMN · Modelo C4 · Repositorio

Versión 1.0 — prototipo funcional implementado y probado

Proyecto individual
Autora: Nasly Trujillo
Docente: Leonardo Valenzuela
Ingeniería de Software · Fundación Universitaria UNINPAHU · Bogotá

Control del documento

Campo	Detalle
Nombre	FastParking — Documento técnico de diseño e implementación
Versión	1.0 (reemplaza al borrador 0.9 del 16 de octubre de 2026)
Fecha	20 de octubre de 2026
Autora	Nasly Trujillo — proyecto individual
Estado	Prototipo implementado, con pruebas automatizadas aprobadas y ejecución local con Docker
Repositorio	[Completar con la URL de GitHub] — el docente tiene acceso como colaborador
Alcance	Disponibilidad, retención de 7 min, reasignación, mapa con ruta y pago son **simulados**; no hay integración con sensores, talanqueras ni pasarelas de pago

Cambios frente a la versión 0.9

- El MER gráfico y el diccionario ahora coinciden y son iguales al esquema SQL del repositorio (7 + 1 entidades con función clara).
- Un único catálogo de estados de la retención: VIGENTE, CANCELADA, EXPIRADA, REASIGNADA, INVALIDADA y UTILIZADA.
- La reasignación automática y la llegada confirmada ahora tienen requerimientos propios (RF-15 y RF-16); se eliminaron rutas con grafos, administrador y auditoría genérica.
- El pago pasa a ser un **pago simulado desde la app** (RF-11, RF-17) con tarifa ilustrativa; se agregó la entidad PAGO.
- Stack "utilizado" (ya no "propuesto"), C4 de 4 niveles derivado del código real, BPMN formal (.bpmn) y matriz de trazabilidad hasta el código y las pruebas.

Contenido

1. Resumen ejecutivo y alcance 5
1.1 Alcance incluido (implementado) 5
1.2 Fuera del alcance 5
1.3 Términos clave 5
2. Problema, objetivos y actores 6
2.1 Problema 6
2.2 Objetivo general 6
2.3 Objetivos específicos 6
2.4 Actores 6
3. Requerimientos del sistema 7
3.1 Requerimientos funcionales 7
3.2 Requerimientos no funcionales 8
4. Reglas de negocio y restricciones 10
4.1 Reglas de negocio 10
4.2 Restricciones 10
5. Stack tecnológico utilizado y justificación 11
6. Arquitectura propuesta 12
6.1 Por qué esta arquitectura responde al problema 12
6.2 Contenedores y responsabilidades 13
6.3 Componentes del backend 13
6.4 Contratos de comunicación (API REST y SSE) 13
6.5 Control de concurrencia 14
7. Modelo Entidad-Relación (MER) 15
7.1 Diagrama 16
7.2 Diccionario de entidades 17
7.3 Relaciones y cardinalidades 17
7.4 Entidades retiradas respecto a la versión 0.9 18
8. BPMN del proceso principal 19
8.1 Elementos BPMN utilizados 21
8.2 Descripción del flujo 21
9. Modelo C4 22
9.1 Nivel 1 — Contexto 22
9.2 Nivel 2 — Contenedores 22
9.3 Nivel 3 — Componentes de la API 24
9.4 Nivel 4 — Código del flujo de retenciones 25
10. Matriz de trazabilidad y pruebas 26
10.1 Matriz de trazabilidad 26
10.2 Casos de prueba automatizados 28
10.3 Resultados 28
10.4 Evaluación de usabilidad (planeada) 28
11. Seguridad, privacidad y riesgos 29
11.1 Privacidad (Ley 1581 de 2012) 29
11.2 Seguridad de la API 29
11.3 Riesgos 29
12. Repositorio y documentación 30
12.1 Estructura 30
12.2 Instalación y ejecución 30
12.3 Variables de entorno 30
12.4 Despliegue 30

13. Decisiones de diseño..... 31
13.1 Limitaciones y trabajo futuro..... 32
Anexo A. Glosario..... 33
Anexo B. Referencias..... 33
Anexo C. Capturas del prototipo..... 34
Lista de verificación de la entrega..... 35

1. Resumen ejecutivo y alcance

FastParking es una aplicación web progresiva (PWA) para celular dirigida a los visitantes del Centro Comercial Parque La Colina (Bogotá). El conductor escanea el QR impreso en el tiquete físico de ingreso, consulta la disponibilidad por piso y zona en un **mapa 3D del sótano**, elige un puesto libre y obtiene una **retención temporal simulada de 7 minutos**. Si un evento de ocupación deja sin cupo su zona, el sistema la **reasigna automáticamente** a la zona más cercana con cupo. Una flecha recorre la ruta simulada hasta el puesto; al llegar, el conductor confirma que estacionó, la app guarda dónde quedó el vehículo y, al salir, puede **pagar desde la app** (pago simulado) y obtener un QR de salida.

El prototipo **no reemplaza** el sistema actual del parqueadero: el tiquete físico, las cajas y las talanqueras siguen funcionando. Como no existe autorización ni API de los sensores del centro comercial, la ocupación se alimenta con un **simulador** a través de un adaptador desacoplado.

1.1 Alcance incluido (implementado)

- Simulación de la máquina de ingreso con QR real y apertura de la PWA mediante URL/QR (sesión anónima).
- Disponibilidad agregada por piso y zona, con estado textual, hora de actualización y tiempo real (SSE).
- Mapa ilustrativo del sótano en perspectiva 3D, elección de puesto, retención de 7 minutos, cancelación y expiración.
- Procesamiento de eventos de ocupación simulados, reasignación automática o invalidación con aviso al conductor.
- Navegación simulada con flecha, instrucciones de giro y distancia; confirmación de llegada.
- Mi vehículo (hora de entrada y lugar) y Billetera (pago simulado, comprobante con QR de salida e historial).
- API REST, base de datos PostgreSQL, 30 pruebas automatizadas, Docker Compose, CI con GitHub Actions y documentación.

1.2 Fuera del alcance

- Lectura de sensores reales, controladores o sistemas del centro comercial.
- Reserva física garantizada de un cupo, control de talanqueras o sustitución del tiquete físico.
- Pagos reales, pasarelas (PSE, Nequi, adquirentes) o almacenamiento de datos financieros.
- Posicionamiento del conductor en tiempo real dentro del sótano (GPS o balizas).
- Conclusiones de reducción real de tiempos o filas sin una evaluación empírica.

1.3 Términos clave

- **Retención temporal simulada:** registro lógico de una selección durante 7 minutos. No garantiza que el espacio esté libre físicamente.
- **Cupo lógico:** capacidad − vehículos registrados por eventos − retenciones vigentes.
- **Pago simulado:** flujo de pago demostrativo; no mueve dinero ni pide datos de tarjeta o cuenta.

2. Problema, objetivos y actores

2.1 Problema

Entre el ingreso al parqueadero del Centro Comercial Parque La Colina y el momento de estacionar, el conductor no tiene información comprensible sobre qué zonas tienen cupo. La ocupación cambia mientras busca, al salir del centro comercial no siempre recuerda dónde dejó el vehículo y, además, debe hacer fila en las cajas para pagar el parqueadero. FastParking estudia si una interfaz por zonas, una retención lógica temporal con reasignación automática y un plano de referencia ayudan a organizar ese proceso. La posible reducción de tiempos de búsqueda es un **resultado por evaluar**, no un beneficio ya demostrado.

2.2 Objetivo general

Diseñar y desarrollar una aplicación web progresiva (FastParking) que permita a los visitantes del Centro Comercial Parque La Colina **consultar la disponibilidad de parqueadero por zona y piso**, **solicitar retenciones temporales simuladas de 7 minutos** con **reasignación automática** cuando la zona elegida se ocupa, **elegir un puesto en el mapa del sótano y seguir una ruta simulada hasta él**, **registrar la ubicación de su vehículo** y **pagar el parqueadero desde la app (pago simulado)**, como prototipo para evaluar su aporte a la búsqueda de estacionamiento.

2.3 Objetivos específicos

1. Analizar el proceso actual de ingreso, búsqueda de estacionamiento y pago en Parque La Colina para identificar las brechas de información y tiempo.
2. Diseñar la arquitectura de software y el modelo de datos que representen la disponibilidad por zona, las retenciones temporales y el pago simulado.
3. Desarrollar un prototipo funcional (PWA + API) con consulta de disponibilidad, mapa con elección de puesto, retención, reasignación, ubicación del vehículo y pago simulado.
4. Evaluar el prototipo con pruebas automatizadas y pruebas de tareas con usuarios que permitan revisar su funcionamiento y usabilidad.

2.4 Actores

ID	Actor	Responsabilidad	Estado
ACT-01	Conductor / visitante	Abre la PWA con el QR, consulta zonas, solicita o cancela una retención y guarda su ubicación	Implementado
ACT-02	Operador de pruebas	Ejecuta el simulador para producir escenarios de ocupación durante la demostración	Implementado (CLI con clave)
ACT-03	Simulador de eventos (sistema)	Emite eventos ENTRADA/SALIDA normalizados, etiquetados como SIMULADOR	Implementado
ACT-04	Sensores / controlador del parqueadero (externo)	Fuente real de ocupación	Futuro; sin integración ni autorización
ACT-05	Sistema de tiquetes y pago del centro comercial (externo)	Proceso tradicional de tiquete físico y pago	Fuera de alcance; FastParking lo complementa

3. Requerimientos del sistema

Cada requerimiento funcional tiene un criterio de aceptación verificable y el caso de prueba que lo comprueba (sección 10). Los identificadores se conservan entre versiones; RF-11 se reactivó como pago simulado.

3.1 Requerimientos funcionales

ID	Nombre	Descripción	Criterio de aceptación	Prueba
RF-01	Acceso por QR	Se simula la máquina de ingreso: el tiquete físico trae un QR con un token. Al escanearlo (o con "Simular escaneo") la PWA abre por URL (`?t=<token>`) y crea o reanuda una sesión anónima	Sin token se muestra el tiquete con su QR; al escanearlo se muestra la pantalla de zonas sin instalar una app; el mismo QR reanuda la misma sesión	CP-01
RF-02	Consultar disponibilidad	Lista las zonas agrupadas por piso con capacidad, ocupados, retenidas y cupo lógico	La respuesta incluye piso, zona, cupo, estado y marca de tiempo	CP-02
RF-03	Representación accesible	El estado de cada zona se muestra con texto, símbolo y color	Cada zona muestra "Disponible", "Pocos cupos" o "Llena"; el color nunca es la única señal	CP-02
RF-04	Solicitar retención	El conductor solicita una retención temporal simulada sobre una zona con cupo	Se crea una retención VIGENTE con id, creación y vencimiento a 7 min; el cupo lógico baja en 1	CP-03
RF-05	Expirar retención	El sistema finaliza las retenciones vencidas	Al superar `vence_en` el estado pasa a EXPIRADA y deja de contar en el cupo	CP-04
RF-06	Procesar evento de ocupación	El backend procesa eventos ENTRADA/SALIDA y detecta conflictos con retenciones vigentes	Si ocupados + retenidas > capacidad, las retenciones afectadas cambian de estado y la PWA recibe aviso	CP-05
RF-07	Proponer alternativa	Si la zona solicitada no tiene cupo, se sugieren zonas cercanas con cupo	Respuesta 409 `ZONA_SIN_CUPO` con hasta 3 alternativas ordenadas por cercanía; si no hay, se informa	CP-03
RF-08	Cancelar retención	El conductor cancela su retención vigente	El estado pasa a CANCELADA, el cupo se libera y no puede cancelarse dos veces	CP-07
RF-09	Mi vehículo	Se guarda dónde quedó el vehículo (automático al confirmar la llegada) y la sección *Mi vehículo* muestra solo la hora de entrada y el lugar (piso, zona y puesto)	Se ven hora de entrada y lugar; la ubicación se recupera en la misma sesión	CP-08, CP-10
RF-10	Mapa del sótano, elección de puesto y ruta simulada	Plano ilustrativo en perspectiva 3D de cada sótano (piso gris, zonas con borde de marca, demarcación reflectiva, vehículos según la ocupación). El conductor toca un puesto libre y lo reserva; luego una flecha recorre la ruta (con instrucciones de giro y distancia) desde la rampa hasta el puesto	El plano indica que es referencia y no localización en vivo; la flecha recorre la ruta y anuncia la llegada al puesto	Prueba de interfaz (Playwright / manual)

ID	Nombre	Descripción	Criterio de aceptación	Prueba
RF-11	Pagar desde la app (simulado)	La sección *Billetera* liquida el tiempo desde la entrada con la tarifa ilustrativa, permite elegir método (Tarjeta, Nequi, PSE en modo demostración) y entrega comprobante con QR de salida	Valor = minutos × tarifa; un solo pago por tiquete; comprobante con código y hora límite de salida; no se piden ni guardan datos financieros	CP-11
RF-12	Registrar eventos	Se registran eventos de ocupación y cada cambio de estado de una retención	`evento_ocupacion` e `historial_retencion` guardan id, fecha, tipo, fuente, motivo y resultado	CP-05
RF-13	Frescura de datos	La interfaz muestra la hora de actualización y advierte si los datos están desactualizados	Se ve "Actualizado a las hh:mm:ss"; tras 45 s sin datos aparece la advertencia	CP-09 / interfaz
RF-14	Manejo de errores	Errores comprensibles con formato uniforme	`{error:{code,message}}`; sin conexión no se presentan datos antiguos como actuales	CP-09
RF-15	Reasignación automática	Si un evento deja sin cupo la zona de una retención vigente, el sistema la reasigna a la zona con cupo más cercana	La retención original pasa a REASIGNADA y nace una VIGENTE en la zona alternativa (mismo piso primero); si no hay alternativa pasa a INVALIDADA	CP-05
RF-16	Confirmar llegada	Al terminar la ruta, el conductor confirma que estacionó	La retención pasa a UTILIZADA, la zona suma un vehículo (evento con fuente CONDUCTOR), el cupo lógico se mantiene y se guarda la ubicación con el puesto	CP-10
RF-17	Historial de pagos	La *Billetera* muestra los pagos realizados	El pago aparece con fecha, valor, tiempo y método	CP-11

3.2 Requerimientos no funcionales

Los umbrales numéricos (por ejemplo RNF-02) son metas de diseño para las pruebas, no resultados medidos.

ID	Atributo	Requerimiento verificable
RNF-01	Usabilidad	Las tareas principales se completan con instrucciones mínimas (prueba de tareas con usuarios)
RNF-02	Rendimiento	Meta: 95 % de consultas en menos de 2 s en el entorno de prueba (meta, no resultado medido)
RNF-03	Consistencia	Dos solicitudes concurrentes por el último cupo nunca quedan ambas VIGENTES (bloqueo de fila + índice único)
RNF-04	Actualización	La PWA recibe cambios por SSE y, como respaldo, consulta cada 15 s
RNF-05	Seguridad	Validación en backend (Zod), endpoint del simulador con clave, sin secretos en el repositorio
RNF-06	Compatibilidad	Navegadores actuales de escritorio y móvil; PWA instalable
RNF-07	Accesibilidad	Texto + símbolo + color, foco visible, nombres accesibles, objetivos táctiles ≥ 44 px
RNF-08	Mantenibilidad	Módulos separados (sesiones, zonas, retenciones, eventos, ubicación) y adaptador de eventos desacoplado
RNF-09	Trazabilidad	Historial de estados con marca de tiempo y motivo; sin datos personales

ID	Atributo	Requerimiento verificable
RNF-10	Resiliencia	Sin conexión se muestra el error; la API nunca se sirve desde caché

4. Reglas de negocio y restricciones

4.1 Reglas de negocio

ID	Regla
RN-01	La disponibilidad se modela por zona; no se representan cupos individuales
RN-02	Toda retención tiene vencimiento: 7 minutos desde su creación (flujo de 7 min, configurable con RETENTION_MINUTES)
RN-03	La retención no garantiza ocupación física ni prioridad frente al proceso tradicional
RN-04	Un evento válido que deja una zona sin cupo obliga a resolver las retenciones afectadas
RN-05	Solo se sugieren o asignan zonas con cupo lógico > 0 en los datos actuales
RN-06	Solo una retención VIGENTE cambia de estado; CANCELADA, EXPIRADA, REASIGNADA, INVALIDADA y UTILIZADA son finales
RN-07	La concurrencia se valida en backend y base de datos, nunca solo en el navegador
RN-08	El pago es simulado: no procesa dinero real, no pide ni almacena datos financieros; complementa las cajas habituales
RN-09	El plano es de referencia; no representa posicionamiento en vivo
RN-10	FastParking complementa el tiquete físico y no reemplaza el proceso del parqueadero
RN-11	Una sesión puede tener como máximo una retención VIGENTE
RN-12	Cupo lógico = capacidad − ocupados (eventos) − retenciones vigentes, nunca negativo
RN-13	Si un evento genera exceso, ceden primero las retenciones más recientes
RN-14	Orden de cercanía: mismo piso y menor diferencia de `orden_cercania`; luego pisos vecinos
RN-15	Un evento repetido (mismo `id_evento`) no se aplica dos veces; un evento más antiguo que el último aplicado se descarta
RN-16	Cobro: minutos completos o fracción desde la entrada (mínimo 1) × tarifa ilustrativa por minuto; un pago por tiquete; la salida se habilita 15 min después del pago

4.2 Restricciones

- Se desconocen fabricante, controlador, protocolo y API de los indicadores del parqueadero; no hay autorización del centro comercial.
- Los datos de pisos, zonas y capacidades son **ilustrativos**.
- El pago es una simulación: no se conectan pasarelas reales (PSE, Nequi, adquirentes) ni se manejan datos financieros.
- La tarifa por minuto es ilustrativa y configurable; no es la tarifa oficial del centro comercial.
- La PWA está diseñada solo para celular.
- No se recolectan placas, identidad ni ubicación GPS (Ley 1581 de 2012).
- No se puede afirmar reducción real de tiempos sin una evaluación con datos comparables.
- Proyecto individual con plazo académico: el alcance prioriza el flujo principal.

5. Stack tecnológico utilizado y justificación

Las tecnologías siguientes están **implementadas** en el repositorio (no son una propuesta). Las versiones exactas quedan en los archivos package.json.

Categoría	Tecnología utilizada	Justificación (necesidad del proyecto)
Lenguaje	TypeScript 5 (frontend y backend)	Un solo lenguaje tipado para una sola desarrolladora; comparte los nombres de estados y contratos entre la PWA y la API y detecta errores antes de ejecutar.
Frontend	React 18 + Vite 5	Interfaz por componentes (zonas, mapa, billetera) con estado reactivo para la cuenta regresiva y los avisos en tiempo real; Vite compila rápido y genera archivos estáticos.
PWA	vite-plugin-pwa (manifest + service worker)	El conductor llega desde un QR: la app se abre en el navegador sin instalar nada (RF-01). El service worker solo guarda la interfaz; la API nunca se sirve desde caché (RNF-10).
Backend	Node.js 22 + Express 4	API REST ligera organizada como monolito modular; soporta Server-Sent Events de forma nativa para notificar reasignaciones y expiraciones.
Validación	Zod	Valida en el servidor todo lo que llega (cuerpos, parámetros, eventos del simulador) y produce errores comprensibles (RNF-05, RF-14).
Base de datos	PostgreSQL 16	El sistema maneja información relacionada (pisos, zonas, sesiones, retenciones, eventos, pagos) que exige integridad referencial, restricciones CHECK y transacciones con bloqueo de filas (SELECT … FOR UPDATE) para que dos conductores no obtengan el último cupo (RNF-03).
Acceso a datos	Driver pg con SQL parametrizado y migraciones SQL	El SQL explícito hace visible el control de concurrencia y el esquema coincide 1:1 con el MER; las migraciones versionadas reconstruyen la base en cualquier equipo.
Tiempo real	Server-Sent Events + sondeo cada 15 s	La ocupación cambia sin acción del conductor; SSE es unidireccional y suficiente, y el sondeo cubre caídas de conexión.
Otras librerías	qrcode-generator	Genera el QR del tiquete simulado y el QR de salida del comprobante.
Pruebas	Vitest + Supertest	Pruebas unitarias de las reglas y de integración de la API contra PostgreSQL real, incluida la prueba de concurrencia.
Simulador	CLI en Node.js (simulator/simular.mjs)	Genera eventos de ocupación controlados y repetibles sin hardware (RF-06).
Servicios externos / APIs	Ninguno	No hay autorización del centro comercial ni de pasarelas de pago; el pago y los eventos son simulados por diseño.
Servicios en la nube	Ninguno en esta versión	Se evita costo y dependencia de cuentas; el despliegue en nube (p. ej. Render + Neon) queda como mejora futura.
Despliegue	Docker + Docker Compose; nginx	Un solo comando levanta base de datos, API y PWA en cualquier equipo, incluido el del docente; nginx sirve la PWA y redirige /api.
Control de versiones	Git + GitHub	Historial de commits, revisión del docente y entrega del código.
Integración continua	GitHub Actions	En cada push verifica tipos y ejecuta las pruebas con un PostgreSQL de servicio.
Diagramas	Graphviz (MER, C4) y BPMN 2.0 (bpmn.io / Camunda Modeler)	Diagramas generados como código desde el mismo modelo del repositorio, editables y reproducibles.

6. Arquitectura propuesta

FastParking usa una arquitectura **cliente-servidor con backend en monolito modular** y un **adaptador de eventos desacoplado**. La PWA solo presenta la información; las reglas de cupo, vencimiento, concurrencia, reasignación y cobro se ejecutan en la API y se garantizan en PostgreSQL. El simulador envía eventos por el mismo contrato que en el futuro usaría un adaptador de sensores reales.

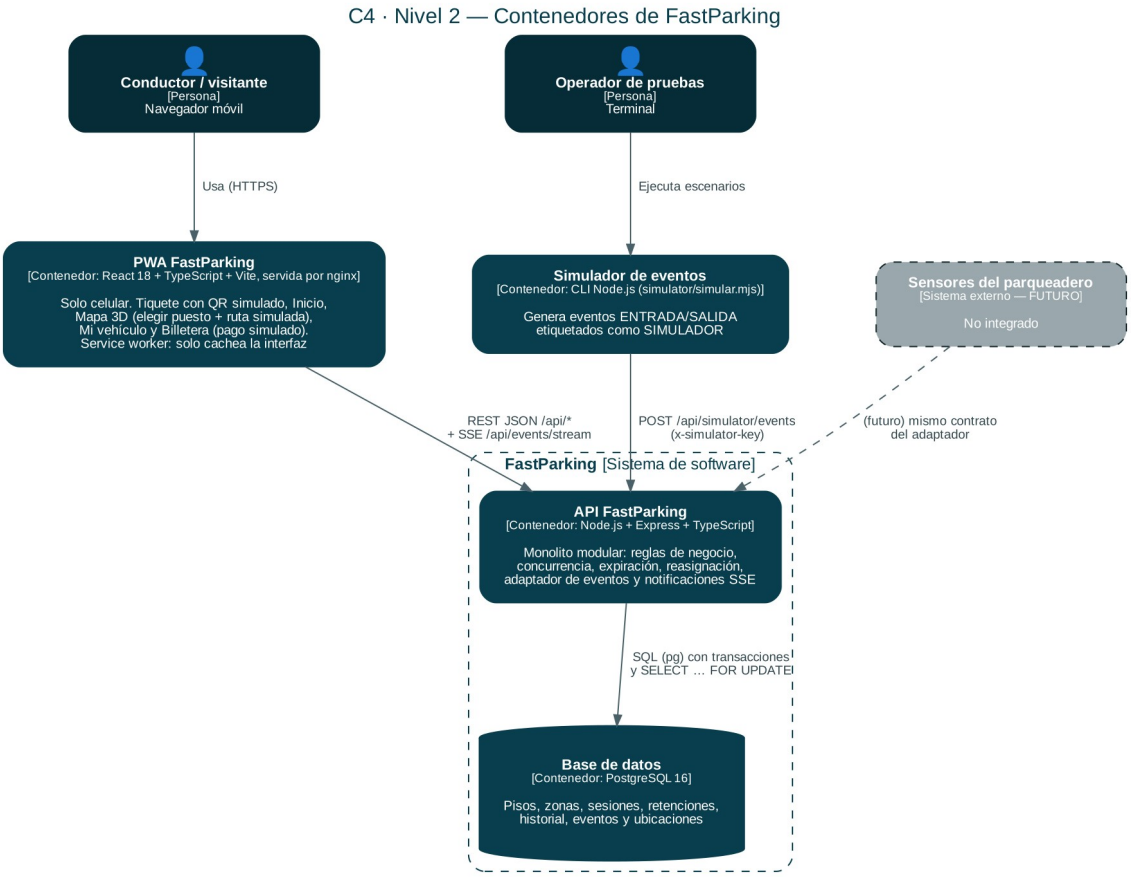


Figura 1. Arquitectura general: actores, contenedores, sistemas externos y comunicación (C4 nivel 2).

6.1 Por qué esta arquitectura responde al problema

Decisión	Por qué responde al problema
PWA en el navegador	El conductor llega desde un QR en el tiquete: no puede instalar una app en la entrada del parqueadero
Monolito modular (un solo despliegue)	Un proyecto individual con plazo corto: un despliegue, una base de datos y módulos con límites claros, sin el costo operativo de microservicios
Reglas en la API y en PostgreSQL	La concurrencia (dos conductores por el último cupo) solo se resuelve de forma confiable en el servidor con transacciones y bloqueo de filas
Adaptador de eventos	No se conoce el fabricante de los sensores: el dominio depende de un contrato (`FuenteOcupacion`), no de una marca
SSE + sondeo	La ocupación cambia sin que el conductor haga nada: el servidor debe empujar los cambios (reasignación, expiración)

6.2 Contenedores y responsabilidades

Contenedor	Tecnología	Responsabilidad	No debe hacer
PWA FastParking	React 18 + TypeScript + Vite, nginx (solo celular)	Tiquete con QR simulado, Inicio (zonas), Mapa 3D con elección de puesto y navegación simulada, Mi vehículo y Billetera (pago simulado e historial)	Decidir si una retención es válida o resolver concurrencia
API FastParking	Node.js + Express + TypeScript	Validar, aplicar reglas, transacciones, expiración, reasignación, SSE	Exponer secretos o confiar en estados calculados en el cliente
Base de datos	PostgreSQL 16	Persistir y garantizar integridad (FK, CHECK, índice único parcial)	Guardar datos personales o financieros
Simulador	CLI Node.js	Generar eventos ENTRADA/SALIDA etiquetados como SIMULADOR	Presentarse como sensor real

6.3 Componentes del backend

Componente	Carpeta	Responsabilidad
Capa HTTP	`app.ts`, `*.routes.ts`, `shared/errors.ts`	Rutas, validación con Zod, sesión requerida, formato de error
Sesiones	`modules/sessions`	Sesión anónima desde el token del QR
Disponibilidad	`modules/zones`	Cupo lógico, estado textual, bloqueo de zona
Retenciones	`modules/retentions/retentions.service.ts`	Solicitar, cancelar, consultar
Motor de reasignación	`modules/retentions/reassignment.service.ts`	Zona alternativa más cercana con cupo
Planificador de expiraciones	`modules/retentions/expiration.scheduler.ts`	Expira retenciones cada 15 s
Adaptador de eventos	`modules/events/event.adapter.ts`	Normaliza mensajes de la fuente
Procesador de eventos	`modules/events/events.service.ts`	Idempotencia, orden, ocupación, conflictos
Ubicación	`modules/locations`	Guardar/recuperar ubicación
Pagos	`modules/payments`	Liquidación por minuto, pago simulado con bloqueo de la sesión, comprobante e historial
Notificador	`shared/realtime.ts`	Server-Sent Events
Acceso a datos	`db/pool.ts`, `*.repository.ts`	SQL parametrizado y transacciones

6.4 Contratos de comunicación (API REST y SSE)

Interfaz	Operación	Descripción
REST	`POST /api/sessions`	Crea o reanuda la sesión anónima
REST	`GET /api/zones`	Disponibilidad agregada y marca temporal
REST	`POST /api/retentions`	Solicita retención (transacción + bloqueo)

Interfaz	Operación	Descripción
REST	`GET /api/retentions/current`, `GET /api/retentions/:id`	Estado y vencimiento
REST	`GET /api/sessions/current`	Hora de entrada (RF-09)
REST	`GET /api/payments/quote · POST /api/payments · GET /api/payments`	Liquidación, pago simulado e historial (RF-11, RF-17)
REST	`POST /api/retentions/:id/arrive`	Confirma llegada: UTILIZADA + ocupación + ubicación (RF-16)
REST	`POST /api/retentions/:id/cancel`	Cancela (se conserva en el historial; por eso no es DELETE)
REST	`PUT/GET /api/vehicle-location`	Ubicación del vehículo
REST	`POST /api/simulator/events`	Evento simulado (cabecera `x-simulator-key`)
SSE	`GET /api/events/stream?sessionId=`	Eventos `zonas` y `retencion`

Las rutas protegidas exigen la cabecera X-Session-Id. Todos los errores usan el formato { error: { code, message, details? } }.

6.5 Control de concurrencia

1. La solicitud de retención abre una transacción y bloquea la fila de la zona con SELECT … FOR UPDATE.
2. Una segunda solicitud por la misma zona espera a que la primera confirme; luego recalcula el cupo y responde 409 ZONA_SIN_CUPO.
3. El índice único parcial uq_retencion_vigente_por_sesion impide dos retenciones vigentes de una misma sesión.
4. Todo el sistema bloquea en el mismo orden (zona → retención) y withTransaction reintenta ante deadlock; el pago bloquea la sesión para no cobrar dos veces.

7. Modelo Entidad-Relación (MER)

El MER corresponde exactamente al archivo backend/db/migrations/001_esquema_inicial.sql. Cada entidad se justifica con al menos un requerimiento.

7.1 Diagrama

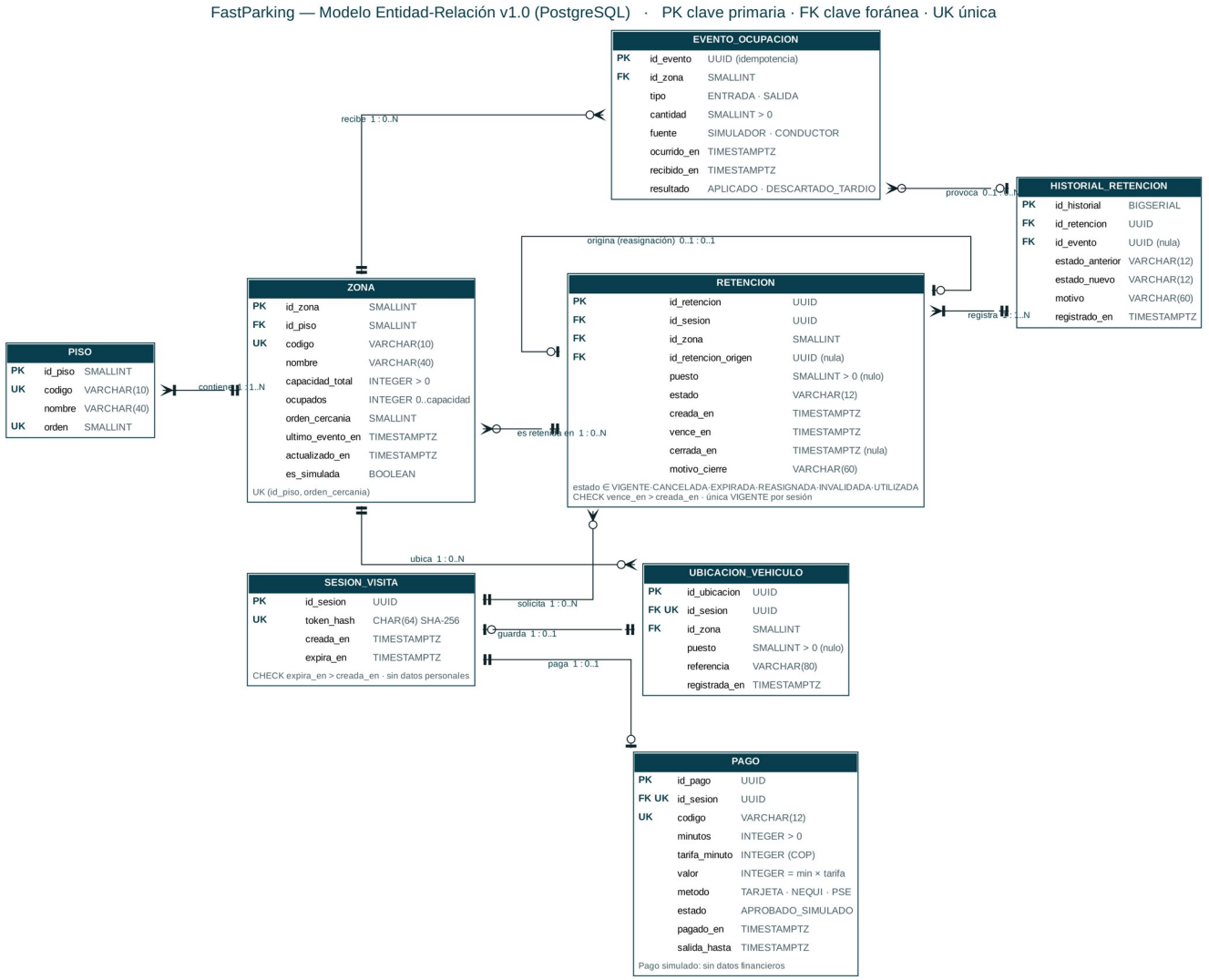


Figura 2. MER v1.0 con claves primarias (PK), foráneas (FK), únicas (UK) y cardinalidades (notación pata de gallo).

7.2 Diccionario de entidades

Entidad	Atributos principales	Claves	Restricciones	Requerimientos
PISO	id_piso, codigo, nombre, orden	PK id_piso · UK codigo · UK orden	NOT NULL	RF-02, RF-10, RF-15
ZONA	id_zona, id_piso, codigo, nombre, capacidad_total, ocupados, orden_cercania, ultimo_evento_en, actualizado_en, es_simulada	PK id_zona · FK id_piso → PISO · UK codigo · UK (id_piso, orden_cercania)	capacidad_total > 0 · 0 ≤ ocupados ≤ capacidad_total	RF-02, RF-03, RF-06, RF-13, RF-15
SESION_VISITA	id_sesion, token_hash, creada_en, expira_en	PK id_sesion · UK token_hash	expira_en > creada_en · solo hash SHA-256 del QR	RF-01
RETENCION	id_retencion, id_sesion, id_zona, id_retencion_origen, puesto, estado, creada_en, vence_en, cerrada_en, motivo_cierre	PK id_retencion · FK id_sesion → SESION_VISITA · FK id_zona → ZONA · FK id_retencion_origen → RETENCION	estado ∈ {VIGENTE, CANCELADA, EXPIRADA, REASIGNADA, INVALIDADA, UTILIZADA} · vence_en > creada_en · cerrada_en nula ⇔ VIGENTE · índice único parcial: una VIGENTE por sesión	RF-04, RF-05, RF-08, RF-15, RF-16
HISTORIAL_RETENCION	id_historial, id_retencion, id_evento, estado_anterior, estado_nuevo, motivo, registrado_en	PK id_historial · FK id_retencion → RETENCION · FK id_evento → EVENTO_OCUPACION (nula)	NOT NULL en estado_nuevo y motivo	RF-12, RNF-09
EVENTO_OCUPACION	id_evento, id_zona, tipo, cantidad, fuente, ocurrido_en, recibido_en, resultado	PK id_evento (lo asigna la fuente → idempotencia) · FK id_zona → ZONA	tipo ∈ {ENTRADA, SALIDA} · cantidad > 0 · fuente ∈ {SIMULADOR, CONDUCTOR} · resultado ∈ {APLICADO, DESCARTADO_TARDIO}	RF-06, RF-12
UBICACION_VEHICULO	id_ubicacion, id_sesion, id_zona, puesto, referencia, registrada_en	PK id_ubicacion · FK+UK id_sesion → SESION_VISITA · FK id_zona → ZONA	puesto > 0 · referencia ≤ 80 caracteres · sin GPS	RF-09, RF-16
PAGO	id_pago, id_sesion, codigo, minutos, tarifa_minuto, valor, metodo, estado, pagado_en, salida_hasta	PK id_pago · FK+UK id_sesion → SESION_VISITA · UK codigo	valor = minutos × tarifa_minuto · metodo ∈ {TARJETA, NEQUI, PSE} · estado = APROBADO_SIMULADO · salida_hasta > pagado_en · sin datos financieros	RF-11, RF-17

7.3 Relaciones y cardinalidades

Relación	Cardinalidad	Significado
PISO contiene ZONA	1 : 1..N	Cada zona pertenece a un piso
ZONA es retenida en RETENCION	1 : 0..N	Histórico de retenciones; las reglas controlan las vigentes
SESION_VISITA solicita RETENCION	1 : 0..N	Máximo una VIGENTE a la vez (RN-11)
RETENCION origina RETENCION	0..1 : 0..1	Una reasignación crea exactamente una retención nueva

Relación	Cardinalidad	Significado
RETENCION registra HISTORIAL_RETENCION	1 : 1..N	Cada cambio de estado deja constancia
EVENTO_OCUPACION provoca HISTORIAL_RETENCION	0..1 : 0..N	Cambios causados por un evento
ZONA recibe EVENTO_OCUPACION	1 : 0..N	Eventos de prueba a lo largo del tiempo
SESION_VISITA guarda UBICACION_VEHICULO	1 : 0..1	Una ubicación por sesión (se actualiza)
ZONA ubica UBICACION_VEHICULO	1 : 0..N	Zona donde quedó el vehículo
SESION_VISITA paga PAGO	1 : 0..1	Un pago simulado por tiquete

7.4 Entidades retiradas respecto a la versión 0.9

Entidad v0.9	Motivo
ESTADO_ZONA	Se integró en ZONA (`ocupados`, `actualizado_en`); las retenidas se calculan, no se duplican
ADMINISTRADOR	Sin RF que la justifique (D-07)
AUDITORIA	Reemplazada por HISTORIAL_RETENCION + EVENTO_OCUPACION (D-10)
PAGO_SIMULADO	Reemplazada por PAGO, ligada a la sesión del tiquete y no a la retención (D-08 revisado)
NODO_PLANO, TRAMO_PLANO	Sin rutas ni Dijkstra (D-05, D-06)

8. BPMN del proceso principal

Proceso modelado: **solicitud de retención temporal simulada y resolución de conflictos por ocupación**. Es un proceso real del sistema, que incluye decisiones del servidor, esperas por eventos y la interacción con el simulador. No es una secuencia de pantallas. El archivo editable es docs/diagramas/bpmn-retencion.bpmn (BPMN 2.0, se abre en bpmn.io o Camunda Modeler).

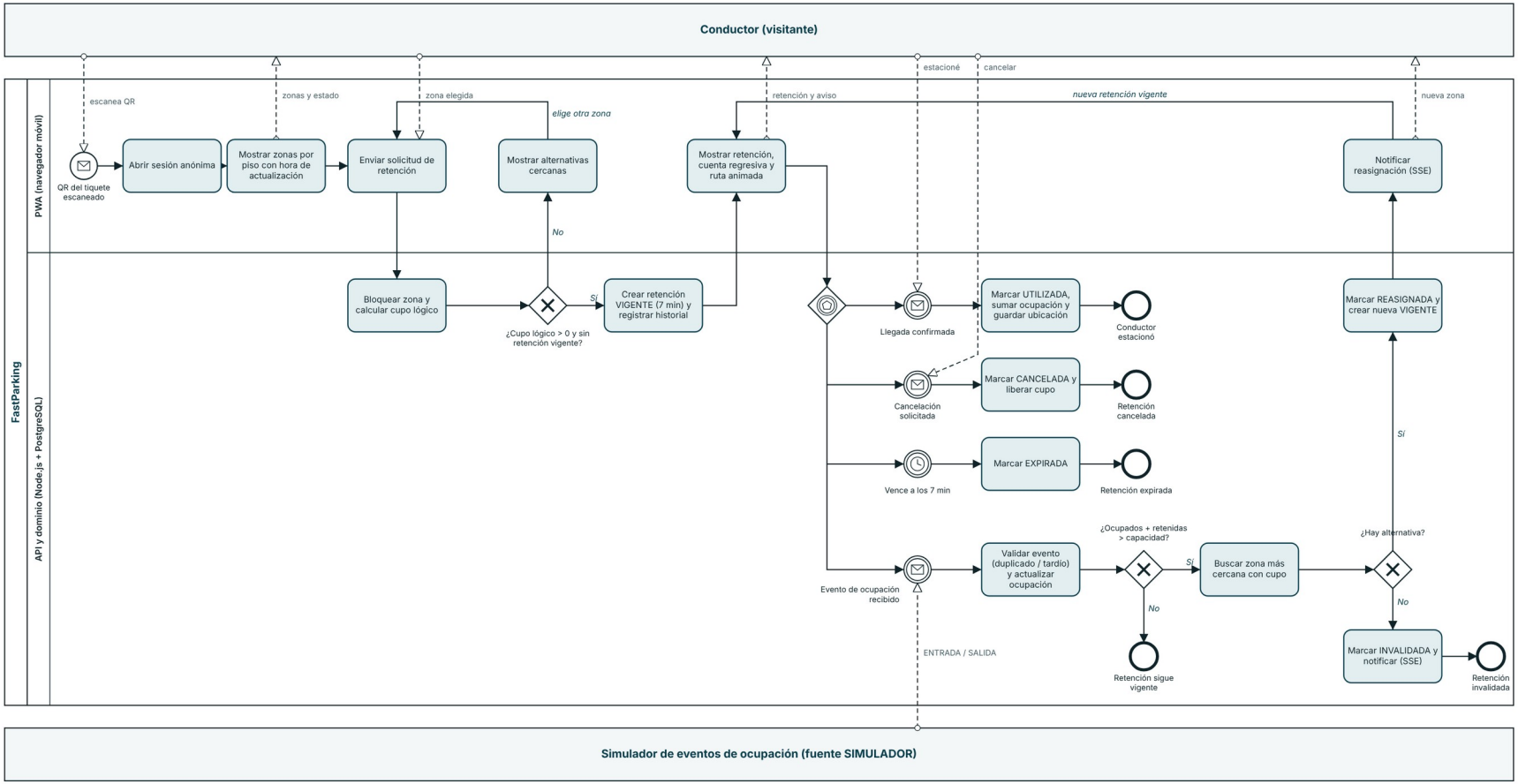


Figura 3. BPMN del proceso de retención y resolución de conflictos (pools, lanes, eventos, gateways y flujos de mensaje).

8.1 Elementos BPMN utilizados

Elemento BPMN	Uso en el proceso
Pools	Conductor (colapsado), FastParking (expandido) y Simulador de eventos de ocupación (colapsado).
Lanes	PWA (navegador móvil) y API y dominio (Node.js + PostgreSQL) dentro del pool FastParking.
Evento de inicio de mensaje	QR del tiquete escaneado.
Gateway exclusivo	¿Cupo lógico > 0 y sin retención vigente?; ¿Ocupados + retenidas > capacidad?; ¿Hay alternativa?
Gateway basado en eventos	Espera lo que ocurra primero: llegada confirmada, cancelación, vencimiento o evento de ocupación.
Eventos intermedios	Mensaje: llegada confirmada, cancelación solicitada, evento de ocupación. Temporizador: vence a los 7 min (PT7M).
Eventos de fin	Conductor estacionó, retención cancelada, retención expirada, retención sigue vigente, retención invalidada.
Flujos de mensaje	Interacciones entre el conductor, la PWA, la API y el simulador (escaneo, zona elegida, avisos, eventos ENTRADA/SALIDA).

8.2 Descripción del flujo

1. El conductor escanea el QR; la PWA abre una sesión anónima y muestra las zonas con su hora de actualización.
2. El conductor elige una zona o un puesto; la API bloquea la zona y calcula el cupo lógico.
3. Si no hay cupo, la PWA muestra alternativas cercanas y el conductor elige otra. Si hay cupo, se crea una retención VIGENTE de 7 minutos y se registra en el historial.
4. La PWA muestra la cuenta regresiva y la ruta animada. El proceso espera el primer evento: llegada confirmada (UTILIZADA, suma la ocupación y guarda la ubicación), cancelación (CANCELADA), vencimiento (EXPIRADA) o evento de ocupación.
5. Ante un evento de ocupación, la API descarta duplicados o eventos tardíos y actualiza la ocupación. Si la zona queda sin cupo, busca la zona más cercana: si existe, la retención pasa a REASIGNADA y nace una nueva VIGENTE que se notifica por SSE; si no, pasa a INVALIDADA.

El pago simulado (RF-11) es un proceso posterior al estacionamiento: liquidación por minuto, selección de método, comprobante y QR de salida.

9. Modelo C4

Los cuatro niveles se derivan del código del repositorio: cada caja del nivel 3 es una carpeta de backend/src y el nivel 4 muestra archivos y funciones reales.

9.1 Nivel 1 — Contexto

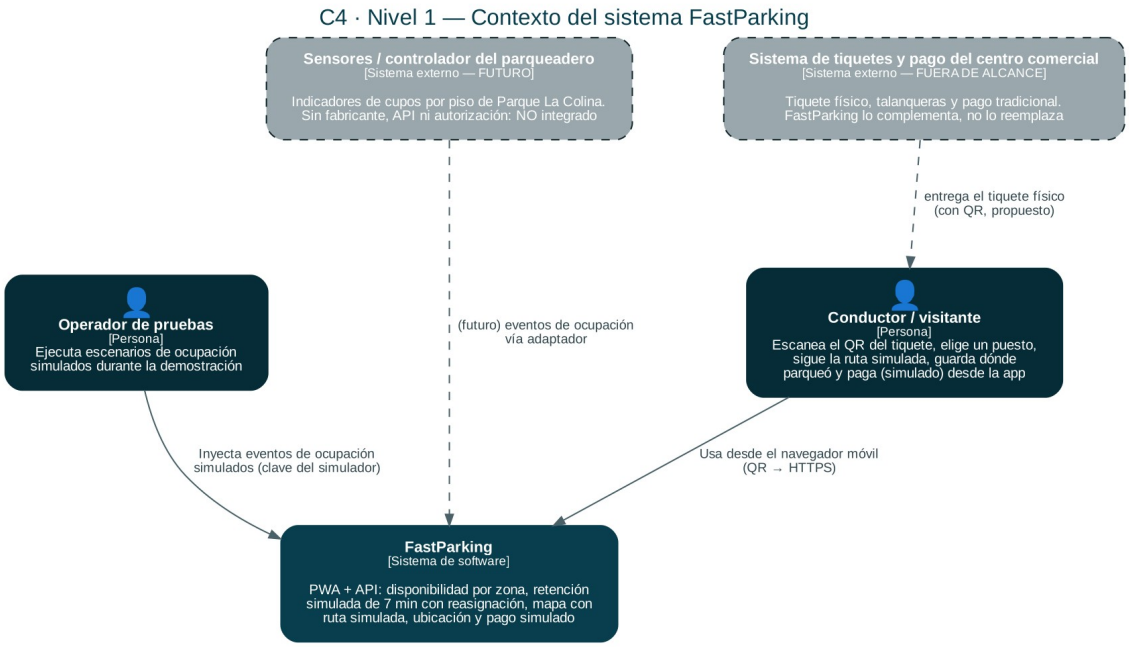


Figura 4. C4 nivel 1: FastParking, sus usuarios y los sistemas externos (futuros o fuera de alcance).

El conductor usa FastParking desde el navegador del celular; el operador de pruebas inyecta eventos simulados. Los sensores del parqueadero (futuro) y el sistema de tiquetes y pago del centro comercial (fuera de alcance) se muestran con línea punteada porque no están integrados.

9.2 Nivel 2 — Contenedores

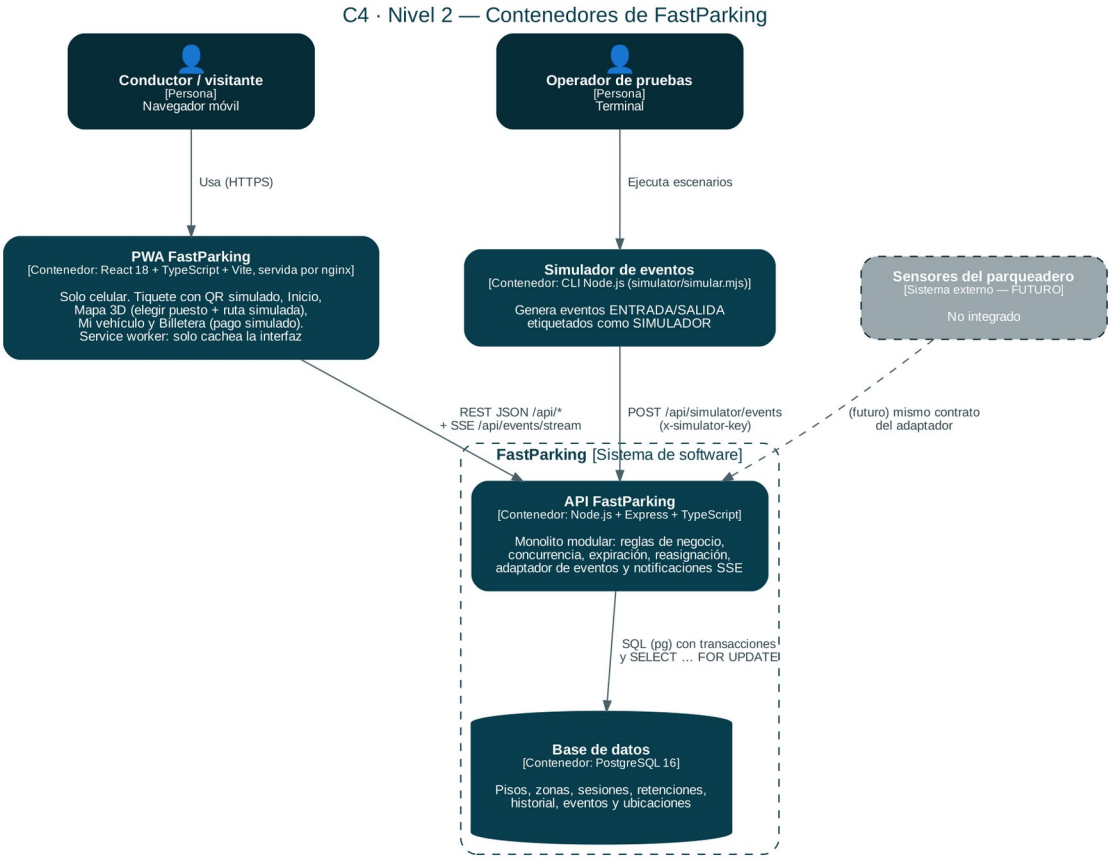


Figura 5. C4 nivel 2: PWA, API, base de datos y simulador.

9.3 Nivel 3 — Componentes de la API

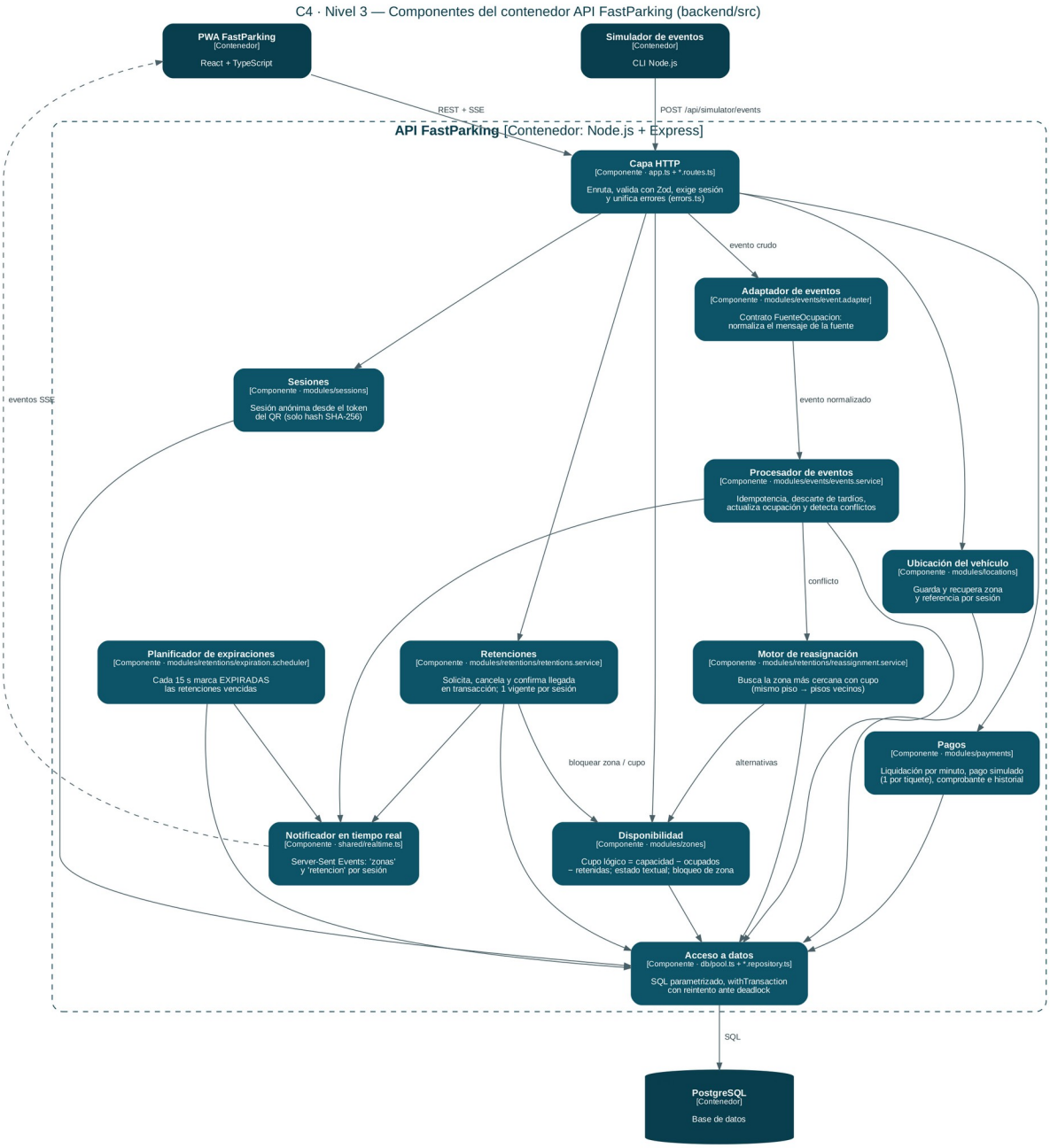


Figura 6. C4 nivel 3: componentes internos del contenedor API y su responsabilidad.

9.4 Nivel 4 — Código del flujo de retenciones

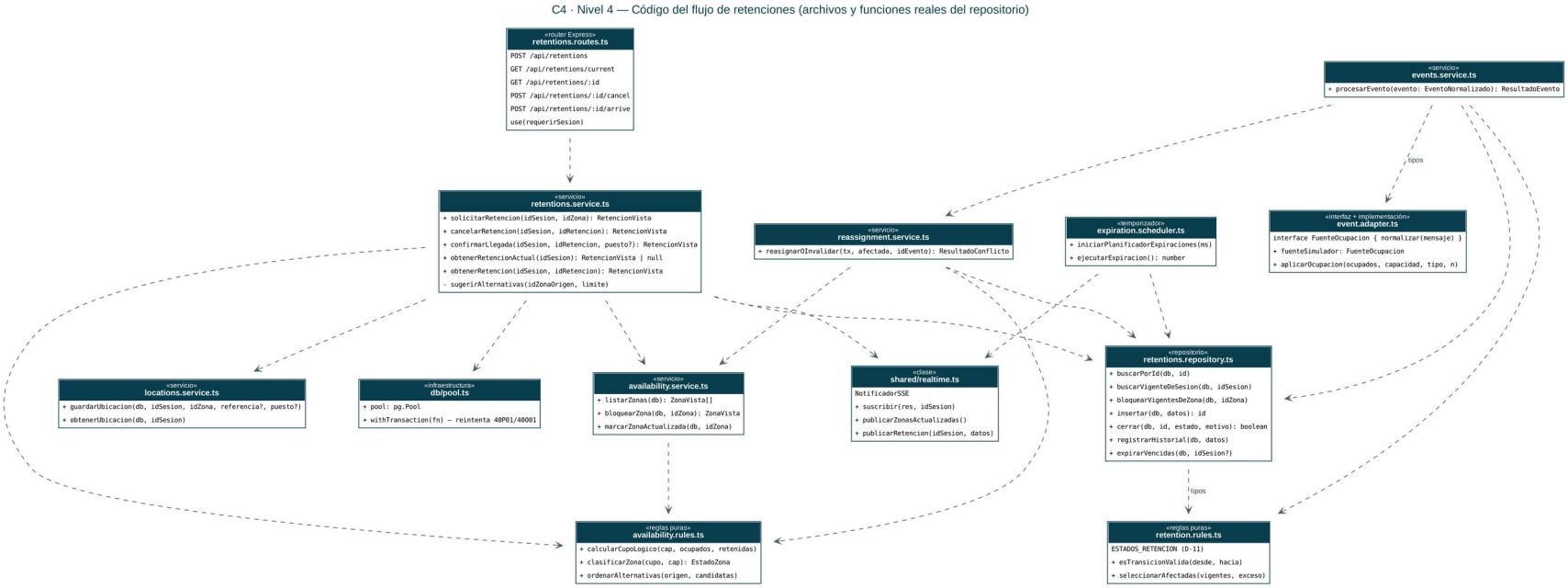


Figura 7. C4 nivel 4: módulos, funciones, interfaz FuenteOcupacion y dependencias reales del repositorio.

10. Matriz de trazabilidad y pruebas

Necesidad → Requerimiento → Proceso (BPMN) → Arquitectura (C4) → Datos (MER) → Código → Prueba. Necesidades: **N-1** incertidumbre sobre dónde hay cupo; **N-2** recordar dónde quedó el vehículo; **N-3** filas para pagar.

10.1 Matriz de trazabilidad

Necesidad	Req.	Tarea / evento BPMN	Componente C4 (nivel 3)	Entidad MER	Código	Prueba
N-1	RF-01	QR del tiquete escaneado · Abrir sesión anónima	Sesiones	SESION_VISITA	`modules/sessions/sessions.service.ts` · `frontend/src/components/Tiquete.tsx` · `hooks/useSesion.ts`	CP-01
N-1	RF-02, RF-03, RF-13	Mostrar zonas por piso con hora de actualización	Disponibilidad	PISO, ZONA	`modules/zones/availability.service.ts` · `availability.rules.ts` · `frontend/src/components/Disponibilidad.tsx` · `hooks/useTiempoReal.ts`	CP-02
N-1	RF-04, RNF-03, RN-07, RN-11	Bloquear zona y calcular cupo lógico · ¿Cupo lógico > 0? · Crear retención VIGENTE	Retenciones · Acceso a datos	RETENCION, HISTORIAL_RETENCION	`retentions.service.ts#solicitarRetencion` · `availability.service.ts#bloquearZona` · `db/pool.ts#withTransaction`	CP-03, CP-06
N-1	RF-07	Mostrar alternativas cercanas	Retenciones · Disponibilidad	ZONA	`retentions.service.ts#sugerirAlternativas` · `availability.rules.ts#ordenarAlternativas` · `PanelAlternativas`	CP-03
N-1	RF-05	Vence a los 7 min → Marcar EXPIRADA	Planificador de expiraciones	RETENCION, HISTORIAL_RETENCION	`expiration.scheduler.ts` · `retentions.repository.ts#expirarVencidas`	CP-04
N-1	RF-08	Cancelación solicitada → Marcar CANCELADA	Retenciones	RETENCION, HISTORIAL_RETENCION	`retentions.service.ts#cancelarRetencion` · `PanelRetencion.tsx`	CP-07
N-1	RF-06, RF-12, RN-15	Evento de ocupación recibido → Validar evento y actualizar ocupación	Adaptador de eventos · Procesador de eventos	EVENTO_OCUPACION, ZONA	`events/event.adapter.ts` · `events/events.service.ts` · `simulator/simular.mjs`	CP-05
N-1	RF-15, RN-13, RN-14	¿Hay alternativa? → Marcar REASIGNADA y crear nueva VIGENTE / Marcar INVALIDADA	Motor de reasignación · Notificador	RETENCION (id_retencion_origen), HISTORIAL_RETENCION	`retentions/reassignment.service.ts` · `shared/realtime.ts` · `App.tsx#alCambiarRetencion`	CP-05

Necesidad	Req.	Tarea / evento BPMN	Componente C4 (nivel 3)	Entidad MER	Código	Prueba
N-1, N-2	RF-10	Mostrar retención, cuenta regresiva y ruta animada	PWA (contenedor)	ZONA, PISO, RETENCION (puesto)	`frontend/src/lib/plano.ts` · `components/MapaSotano.tsx`	Prueba de interfaz
N-1, N-2	RF-16	Llegada confirmada → Marcar UTILIZADA, sumar ocupación y guardar ubicación	Retenciones · Ubicación	RETENCION, EVENTO_OCUPACION (fuente CONDUCTOR), UBICACION_VEHICULO (puesto)	`retentions.service.ts#confirmarLlegada`	CP-10
N-2	RF-09	(resultado de la llegada confirmada)	Ubicación del vehículo · Sesiones	UBICACION_VEHICULO, SESION_VISITA (hora de entrada)	`modules/locations/*` · `GET /api/sessions/current` · `frontend/src/components/MiVehiculo.tsx`	CP-08, CP-10
N-3	RF-11, RF-17, RN-08, RN-16	(proceso de pago, posterior al estacionamiento)	Pagos	PAGO, SESION_VISITA	`modules/payments/*` · `frontend/src/components/Billetera.tsx`	CP-11
—	RF-14, RNF-05, RNF-10	(transversal)	Capa HTTP	—	`shared/errors.ts` · `frontend/src/api.ts` · `EstadoDatos`	CP-09

10.2 Casos de prueba automatizados

ID	Escenario	Resultado esperado	Archivo
CP-01	Health y sesión desde QR	API y BD responden; el mismo QR reanuda la sesión	`backend/tests/api.test.ts`
CP-02	Zonas con estados mixtos	Aparecen DISPONIBLE, POCOS_CUPOS y LLENA con marca de tiempo	idem
CP-03	Solicitar retención	VIGENTE a 7 min, cupo −1; 2.ª retención de la misma sesión → 409; zona llena → 409 con alternativas	idem
CP-04	Vencimiento	EXPIRADA y deja de ser vigente	idem
CP-05	Evento de ocupación	Reasignación a S1-D; invalidación sin alternativas; idempotencia; descarte de tardíos; clave del simulador	idem
CP-06	Concurrencia	Dos conductores por el último cupo: estados [201, 409] y una sola VIGENTE	idem
CP-07	Cancelar	CANCELADA, libera cupo, no se cancela dos veces, otra sesión no puede cancelarla	idem
CP-10	Llegada confirmada	UTILIZADA, ocupados +1, cupo igual al que tenía con la retención, ubicación con puesto, evento CONDUCTOR, no se confirma dos veces	idem
CP-11	Pago simulado e historial	Liquidación por minuto, un solo pago (2.° → 409), método inválido → 400, historial con 1 pago; puesto elegido en el mapa se guarda	idem
CP-08	Ubicación	Se guarda y recupera zona, piso y referencia	idem
CP-09	Errores	401 sin sesión, 400 validación, 404 ruta	idem
Unitarias	Reglas de negocio puras	Cupo lógico, clasificación, orden de alternativas, transiciones, afectadas	`backend/tests/reglas.test.ts`

10.3 Resultados

Ejecución con Vitest contra PostgreSQL (npm test en backend): **2 archivos, 30 pruebas, 30 aprobadas, 0 fallidas**. La verificación de tipos (npm run typecheck) no reporta errores. La evidencia se guarda en docs/pruebas/ y el CI de GitHub Actions repite las pruebas en cada push.

La interfaz se probó con un recorrido completo en vista de celular: escaneo del QR, elección de puesto en el mapa, navegación simulada, confirmación de llegada, Mi vehículo y pago simulado (capturas en el Anexo C).

10.4 Evaluación de usabilidad (planeada)

Prueba formativa de tareas con 6 a 8 voluntarios: (1) escanear el QR y encontrar una zona con cupo, (2) reservar un puesto en el mapa, (3) seguir la ruta y confirmar la llegada, (4) pagar y encontrar el QR de salida. Se registrarán tiempos, errores y observaciones. Esta muestra no es estadísticamente representativa y no permite afirmar reducción de tiempos en el parqueadero real.

11. Seguridad, privacidad y riesgos

11.1 Privacidad (Ley 1581 de 2012)

- Sesión anónima: solo se guarda el hash SHA-256 del token del QR; no se recolectan placa, nombre ni GPS.
- La ubicación del vehículo se limita a piso, zona y puesto, y está ligada a la sesión del tiquete.
- El pago simulado no pide ni almacena tarjetas, cuentas o credenciales.
- Los secretos viven en variables de entorno; el repositorio solo incluye archivos .env.example sin valores sensibles.

11.2 Seguridad de la API

- Validación de entradas con Zod, SQL parametrizado, límite de 10 KB por cuerpo y cabeceras X-Content-Type-Options y Referrer-Policy.
- Formato de error uniforme sin trazas internas.

11.3 Riesgos

Riesgo	Impacto	Mitigación implementada
Datos por zona poco precisos	Pérdida de confianza	Hora de actualización visible, aviso de datos desactualizados y "no garantiza el cupo" en cada retención.
Eventos duplicados o tardíos	Ocupación incorrecta	Idempotencia por id_evento y descarte de eventos más antiguos que el último aplicado (CP-05).
Solicitudes concurrentes	Dos retenciones por el último cupo	Bloqueo de fila, índice único parcial y prueba de concurrencia (CP-06).
Confusión sobre la reserva	El usuario cree que el cupo está garantizado	Texto "Retención temporal simulada" en mapa, panel y API.
Confusión sobre el pago	El usuario cree que se cobró dinero real	Etiqueta "pago simulado" en la billetera y en el comprobante; no se piden datos financieros.
Abuso del simulador	Eventos falsos	Endpoint protegido con SIMULATOR_API_KEY (comparación en tiempo constante).
Dependencia de sensores externos	Sin datos reales	Adaptador desacoplado (FuenteOcupacion) y simulador.

12. Repositorio y documentación

Repositorio: **[completar con la URL de GitHub]**. El docente Leonardo Valenzuela se agrega como colaborador con permiso de lectura. El historial de commits muestra la construcción incremental: estructura, base de datos, API, retenciones, eventos, pruebas, PWA, simulador, documentación y pago.

12.1 Estructura

```
fastparking/
├─ README.md              nombre, integrante, tecnologías, arquitectura, instalación, variables
├─ docker-compose.yml     base de datos + API + PWA
├─ .env.example           variables sin secretos
├─ .github/workflows/     integración continua
├─ backend/               API Node.js + Express + TypeScript
│  ├─ db/                 migraciones SQL (MER ejecutable) y datos ilustrativos
│  ├─ src/modules/        sessions · zones · retentions · events · locations · payments
│  └─ tests/              pruebas unitarias e integración (CP-01 … CP-11)
├─ frontend/              PWA React + Vite
├─ simulator/             simulador de eventos y escenarios
└─ docs/                  documento técnico, diagramas, capturas y anexos en .md
```

12.2 Instalación y ejecución

1. Requisitos: Docker Desktop, Node.js 20.12 o superior y Git.
2. Copiar .env.example como .env (en la raíz y en backend).
3. docker compose up --build y abrir http://localhost:8080 (PWA) — la API responde en http://localhost:3000/api/health.
4. Pruebas: docker compose up -d db; luego en backend: npm install y npm test.
5. Simulador: node simulator/simular.mjs llenar S1-C (con SIMULATOR_API_KEY).

12.3 Variables de entorno

Variable	Descripción
DATABASE_URL / TEST_DATABASE_URL	Conexión a PostgreSQL (aplicación y pruebas)
PORT	Puerto de la API (3000)
SIMULATOR_API_KEY	Clave que exige el endpoint del simulador
RETENTION_MINUTES	Vigencia de la retención (7)
SESSION_HOURS	Vigencia de la sesión anónima (12)
EXPIRATION_INTERVAL_MS	Frecuencia del planificador de expiraciones
TARIFA_POR_MINUTO / MINUTOS_PARA_SALIR	Tarifa ilustrativa (130 COP) y minutos para salir tras pagar (15)
POSTGRES_PASSWORD	Contraseña local del contenedor de PostgreSQL

12.4 Despliegue

Ejecución local con Docker Compose. No hay despliegue público en esta versión.

13. Decisiones de diseño

ID	Decisión	Motivo	Consecuencia
D-01	Stack: React + Vite (PWA), Node.js + Express + TypeScript, PostgreSQL con driver `pg` y SQL explícito, Vitest, Docker	Un solo lenguaje; PostgreSQL ofrece transacciones y bloqueo de filas para la concurrencia	Se descartó Prisma: el SQL explícito hace visible el control de concurrencia y el esquema coincide 1:1 con el MER
D-02	Sesión anónima creada desde el token del QR	Minimiza datos personales y alcance	Solo se guarda el hash SHA-256 del token; sin cuentas ni login
D-03	Retención por zona	No existen datos validados por espacio individual	RN-01, RN-12
D-04	Vigencia de 7 min (flujo de 7 min definido por la autora) y 1 retención vigente por sesión	Evita acaparar cupos y simplifica la concurrencia	Índice único parcial `uq_retencion_vigente_por_sesion`
D-05	Reasignación automática simple (mismo piso → pisos vecinos), sin grafo ni Dijkstra	Conserva la idea original con un alcance realista	RF-15; se eliminaron NODO_PLANO y TRAMO_PLANO
D-06	Plano SVG ilustrativo en perspectiva 3D con ruta simple (rampa → vía → puesto) y flecha animada; sin grafos	No hay planos oficiales autorizados; la flecha comunica la navegación sin simular un GPS real	RF-10, RN-09
D-07	Sin módulo de administrador; el simulador se protege con `SIMULATOR_API_KEY`	No había RF que justificara cuentas de administrador	Se eliminó la entidad ADMINISTRADOR
D-08	Pago **simulado** dentro del alcance (revisado): la autora definió que pagar desde la app es parte central del objetivo de ahorrar tiempo	Sin pasarelas reales ni datos financieros	RF-11, RF-17, entidad PAGO
D-09	SSE con respaldo de sondeo cada 15 s	Comunicación servidor → cliente unidireccional y simple	RNF-04, RNF-10
D-10	Historial de estados por retención en lugar de una tabla AUDITORIA genérica	Trazabilidad sin entidades redundantes	Tabla `historial_retencion`
D-11	Catálogo único de estados: VIGENTE, CANCELADA, EXPIRADA, REASIGNADA, INVALIDADA, UTILIZADA	Coherencia entre documento, BPMN, MER y código	CHECK en la BD y `retention.rules.ts`
D-12	Despliegue local con Docker Compose	Ejecución reproducible para el docente sin costos de nube	Nube como mejora futura
D-13	Objetivo general reescrito sin prometer reducción de filas de pago	Coherencia con el alcance	Ver `requerimientos.md`
D-14	Estado UTILIZADA al confirmar la llegada (RF-16)	El proceso necesitaba un final exitoso; la llegada convierte la retención en ocupación física	Evento de ocupación con fuente CONDUCTOR y ubicación con puesto
D-15	Simulación de la máquina de ingreso con QR real	Demostrar RF-01 sin hardware del centro comercial	Pantalla Tiquete; no se almacena la placa
D-16	Navegación por pestañas: Inicio, Mapa, Mi vehículo, Billetera; el tiquete con QR es la pantalla de entrada	Flujo real: el QR está en la máquina de ingreso; la billetera concentra pago e historial	Diseño solo para celular (marco de teléfono en computador)
D-17	El conductor elige el puesto tocándolo en el mapa	Experiencia directa; la retención sigue siendo por zona (RN-01) y el puesto es ilustrativo	Columna `retencion.puesto`

13.1 Limitaciones y trabajo futuro

- Integrar un adaptador real de sensores, si el centro comercial lo autoriza, con el mismo contrato FuenteOcupacion.
- Pasarela de pago real con certificación (PSE, Nequi) y conciliación con las cajas.
- Planos oficiales del parqueadero y posicionamiento interior.
- Despliegue en la nube y evaluación con usuarios reales.

Anexo A. Glosario

Término	Definición en este proyecto
PWA	Aplicación web progresiva: se abre desde el navegador y puede instalarse; aquí, solo para celular.
Retención temporal simulada	Registro lógico de un puesto/zona por 7 minutos, sin garantía física.
Cupo lógico	Capacidad − ocupados − retenciones vigentes.
Evento de ocupación	Mensaje ENTRADA/SALIDA normalizado que cambia la ocupación de una zona.
Adaptador	Componente que traduce una fuente de eventos al contrato interno común.
SSE	Server-Sent Events: canal del servidor al navegador para notificaciones.
MER / BPMN / C4	Modelo entidad-relación; notación de procesos; modelo de arquitectura por niveles.

Anexo B. Referencias

- Brown, S. (s. f.). The C4 model for visualising software architecture. https://c4model.com/
- Object Management Group. (2011). Business Process Model and Notation (BPMN) Version 2.0. https://www.omg.org/spec/BPMN/2.0/
- Congreso de Colombia. (2012). Ley 1581 de 2012. Protección de datos personales.
- PostgreSQL Global Development Group. (2024). PostgreSQL 16 Documentation: Explicit Locking. https://www.postgresql.org/docs/16/explicit-locking.html
- Schwaber, K., y Sutherland, J. (2020). La Guía de Scrum.

Anexo C. Capturas del prototipo

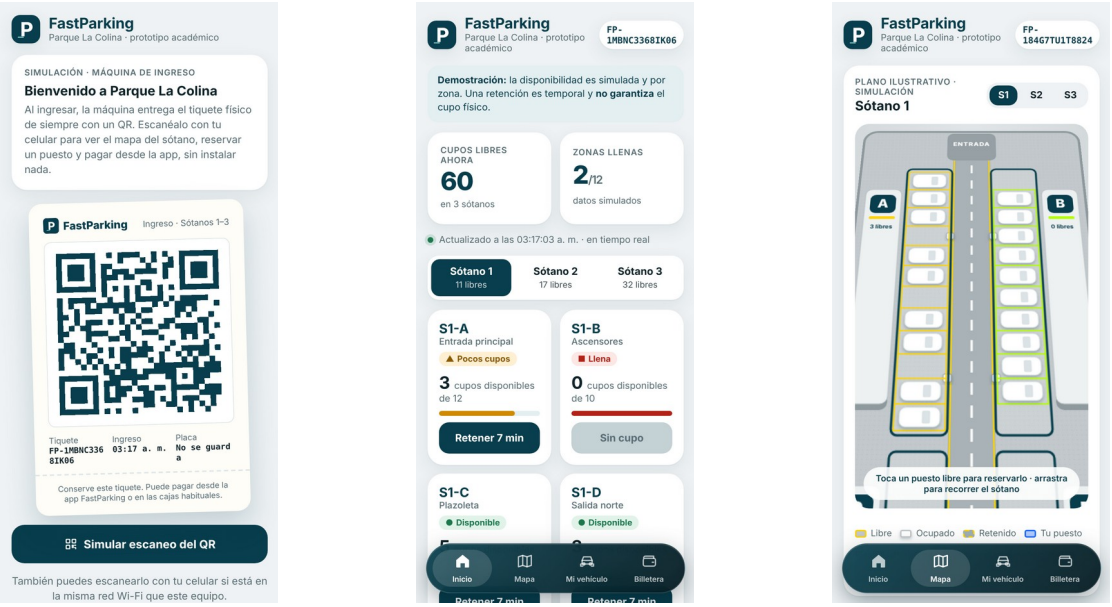


Figura C1. Tiquete con QR simulado · Inicio con zonas por piso · Mapa 3D del sótano.

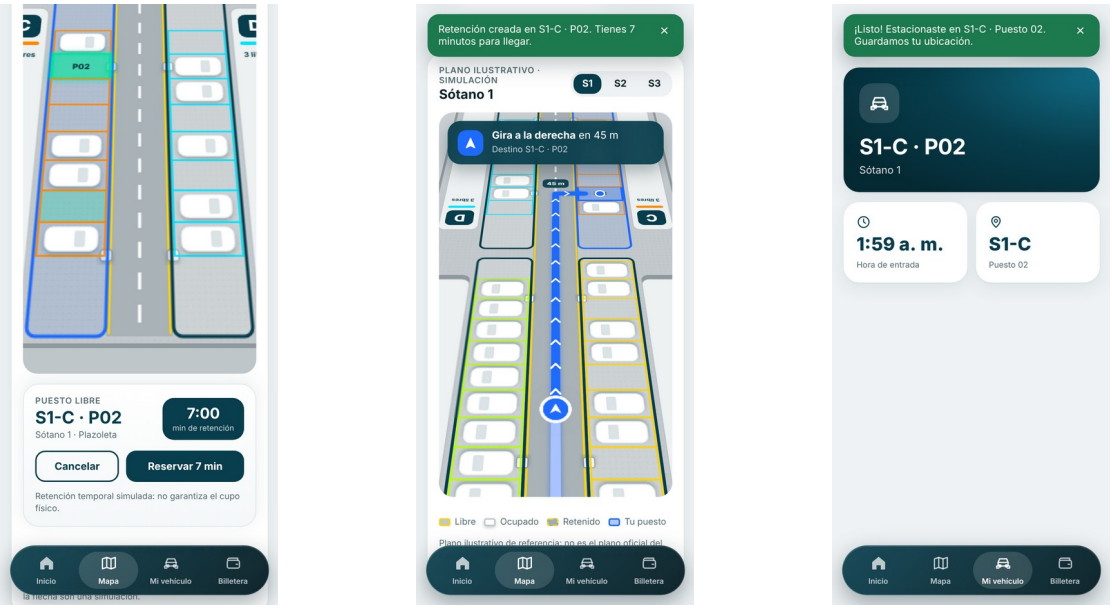


Figura C2. Elección de puesto · Navegación simulada con instrucción de giro · Mi vehículo.

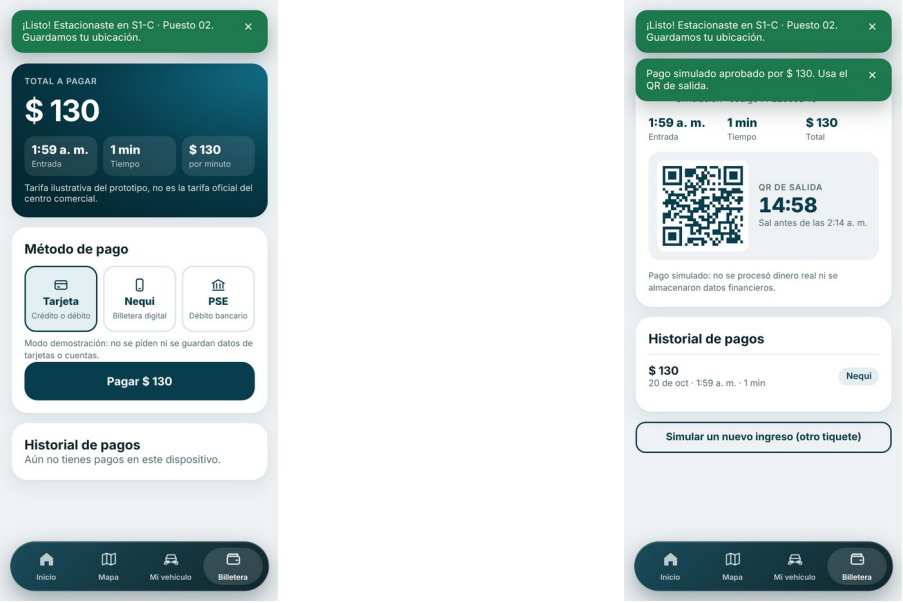


Figura C3. Billetera (pago simulado) · Comprobante con QR de salida e historial.

Lista de verificación de la entrega

- Requerimientos: problema, objetivo, actores, RF, RNF, reglas y restricciones (secciones 2–4).
- Stack utilizado con justificación por categoría (sección 5).
- Arquitectura con actores, contenedores, APIs, sistemas externos, comunicación y justificación (sección 6).
- MER con entidades, atributos, PK, FK, relaciones, cardinalidades y restricciones (sección 7).
- BPMN con pools, lanes, eventos, gateways y flujos de mensaje (sección 8 y archivo .bpmn).
- C4 niveles 1 a 4 coherentes con el código (sección 9).
- Repositorio con código, README, .env.example, historial de commits y acceso del docente (sección 12).
- Trazabilidad necesidad → código → prueba (sección 10).